```
table_schemas = table_schemas.drop(
        "table_catalog",
        "table_schema",
        "ordinal_position",
        "character_octet_length",
        "numeric_precision",
        "numeric_precision_radix",
        "numeric_scale",
        "datetime_precision",
        "interval_type",
        "interval_precision",
        "identity_start",
        "identity_increment",
        "identity_maximum",
        "identity_minimum",
        "identity_cycle",
        "is_system_time_period_start",
        "is_system_time_period_end",
        "system_time_period_timestamp_generation",
        "is_updatable"
    )
```

Figure 8.23 – (L) Collecting the table schema and system information schema; (R) dropping unnecessary and repetitive columns

5. Next, we create two helper functions. The first function organizes the schema information provided, `table_schemas`, creating a table definition. The second collects two rows of data as examples.

```
def table_def(table):
    table_schema = table_schemas.drop("table_name").where(f'table_name = "{table}"')

    return f"Table Schema for {table}: \ncolumn_index" + table_schema.toPandas().to_csv(
        sep="|"
    )

def table_records(table):
    records = spark.sql(f"select * from {catalog}.{schema}.{table} limit 2")

    return f"Example records for {table}: \ncolumn_index" + records.toPandas().to_csv(
        sep="|"
    )
```

Figure 8.24 – Helper functions for organizing table information